

Algorithms for Lattice Fermions User Workshop 2024

15-19th Jul 2024

Pre-conference event: Installation
12th Feb 2024



ALF

Installation notes

ALF Team

July 2024

University of Würzburg

1. Introduction

- The ALF project

- Prerequisites

2. ALF Installation

- Installing ALF prerequisites

- Installing ALF

- Testing ALF

3. pyALF installation

- Testing pyALF

Introduction

The ALF project contains the following public parts:

- **ALF**: our main package, responsible for performing auxiliary-field quantum Monte Carlo simulations; also its **documentation** can be found here
- **pyALF**: provides a Python interface and scripts, as well as Jupyter notebooks
- **ALF_Tutorial**: stores ALF's tutorial, including solutions to the proposed exercises
- **ALF_Presentations**: where, for instance, these notes are made available
- **ALF_code**: the main package's legacy version, ALF 1.0

Version-control is done using Git and they are available in the GitLab repository:

<https://git.physik.uni-wuerzburg.de/ALF>.

These notes cover the installation for ALF and pyALF.

Introduction - Prerequisites

To run ALF and pyALF you need the following installed in your machine:

For ALF:

- Make
- A Fortran compiler,
such as `gfortran` or `ifort`
- Blas library
- Lapack library
- Python3
- MPI libraries (optional)
- For HDF5
 - C++ preprocessor (default: `g++`)
 - Curl
 - gzip development libraries

Install pyALF + Jupyter via pip.

ALF Installation

Installing ALF prerequisites – Linux

To install the required libraries and compiler in one of the following Linux distributions, enter in your terminal:

- Debian/Ubuntu/Linux Mint:

```
sudo apt install make gfortran libblas-dev liblapack-dev \  
python3 libopenmpi-dev g++ curl libghc-zlib-dev git
```

- Red Hat/Fedora/CentOS:

```
sudo yum install make gfortran blas-devel lapack-devel \  
python3 openmpi-devel g++ curl zlib-devel git
```

- OpenSuSE/SLES:

```
sudo zypper install make gcc-fortran lapack-devel \  
python3 openmpi-devel gcc-c++ curl zlib-devel git
```

- Arch Linux:

```
pacman -S make gcc-fortran lapack python3 openmpi curl zlib git
```

MacOS

For MacOS, Xcode and the Apple developer tools should be installed.

gfortran for MacOS can be found at <https://gcc.gnu.org/wiki/GFortranBinaries#MacOS>. Although the most convenient way to get gfortran and MPI libraries set up might be through [Homebrew](#):

```
brew install gfortran open-mpi
```

Windows 10/11

For Windows 10/11, we suggest using the Windows subsystem for Linux to install Ubuntu and the follow the instructions for Ubuntu. For installation instructions see:

<https://docs.microsoft.com/en-us/windows/wsl/install>

Older Windows versions

For older Windows versions, you can use Cygwin, found at <https://www.cygwin.com/>, which provides a Unix-like environment for Windows.

Downloading ALF source code

With its dependencies in place, we can proceed to the proper ALF installation.

ALF's source code can be downloaded by entering the following in a shell:

```
git clone https://git.physik.uni-wuerzburg.de/ALF/ALF.git
```

pyALF can download the ALF source code itself, will download ALF every time it does not find it. Therefore it's more practical to tell pyALF where to find ALF by setting the environment variable `ALF_DIR`.

```
export ALF_DIR="/path/to/ALF"
```

where `/path/to/ALF` is the location of the ALF code, for example `/home/someone/Programs/ALF`. To not have to repeat this command in every terminal session, it is advisable to add it to a file sourced when starting the shell, e.g. `~/.bashrc` or `~/.zshrc`.

To set the necessary environment variables and compiler directives source the script `configure.sh`:

```
source configure.sh [MACHINE] [MODE] [STAB] [Devel|HDF5]
```

where `MACHINE` can be, e.g., `GNU`, for choosing `gfortran` (see ALF's documentation).

When activating HDF5 the first time, it will be downloaded and compiled automatically, which takes several minutes.

In order to compile the libraries, the analysis routines and the QMC program at once, just execute the single command:

```
make
```

When the compilation is over you are in position to run simulations with ALF.

Now let us use ALF:

1. Create a data directory with the content of `Start` found in the folder `Scripts_and_Parameters_files`:

```
cp -r $ALF_DIR/Scripts_and_Parameters_files/Start ./Run && cd ./Run/
```

2. Run an executable Hamiltonian, e.g.:

```
$ALF_DIR/Prog/ALF.out
```

3. Perform default analysis¹:

```
$ALF_DIR/Analysis/ana.out *
```

4. Check results. For instance, `Ener_sca1J` should be around -30 for default parameters.

¹The `analysis.sh` bash script from earlier versions of ALF, run without arguments, is still available in the `Start` directory.

pyALF installation

Warning

In previous versions of pyALF, the installation instructions asked the users to set the environment variable `PYTHONPATH`. This conflicts with the newer pip package, therefore you should remove definitions of this environment variable related to pyALF.

The easiest way to install pyALF with all its Python dependencies is via the Python package installer [pip](#):

```
pip install pyALF
```

If you do not yet have a Python environment to use pip, we recommend [miniforge](#) or [Anaconda](#).

To use the Jupyter Notebooks utilized in the following parts of this tutorial, Jupyter is needed:

```
pip install jupyter
```

Now we can test if pyALF has been successfully installed. pyALF comes with some scripts set up to be run from the Unix shell one of them in `minimal_ALF_run`, runs a very short simulation of the Hubbard model on a square lattice and calculates the energy. Just execute from the shell:

```
minimal_ALF_run
```

After successful execution, there should be a folder `ALF_data`, containing the finished simulation.

Alternative: Ready-to-use container image

For a ready-to-use environment, one can use the Docker image [alfcollaboration/jupyter-pyalf-full](#), which has the above mentioned dependencies, ALF and pyALF installed. With a suitable container runtime e.g. [Docker](#) or [Podman](#), it can be used run ALF and pyALF without any further setup. It is derived from the Jupyter Docker Stacks, therefore [this documentation](#) applies. For example, one could run a container like this:

```
docker run -it --rm -p 127.0.0.1:8888:8888 -v "$PWD":/home/jovyan/work \
    docker.io/alfcollaboration/jupyter-pyalf-full
```

It is also possible to use the container without launching the included JupyterLab. The following command launches a container, which executes `minimal_ALF_run`, saving the results in the current working directory and removing the container right after that.

```
docker run -it --rm -v "$PWD":/home/jovyan/work \
    docker.io/alfcollaboration/jupyter-pyalf-full \
    bash -c 'cd /home/jovyan/work && minimal_ALF_run'
```